Contents

Waveshare Support Report — 10.85inch e-Paper HAT+ (G)

Product: 10.85inch e-Paper HAT+ (G), 1360×480, 4-colour (black/white/red/yellow) **Symptom:** Panel does not display. It painted **once**, partially, and has never repeated despite dozens of attempts in the identical configuration. **Status:** Reproduced after full RMA replacement of both panel and driver HAT.

Summary

The panel accepts SPI commands but never completes power-on. Specifically:

- After `Reset()`, BUSY reads idle.
- On command `0x04` (POWER_ON), BUSY asserts busy.
- **BUSY never releases.** The wait loop runs indefinitely.

This has been reproduced across **2 panels, 2 HAT+ driver boards, 3 Raspberry Pi models, 2 OS installations, and 5 independent software stacks** — including Waveshare's own C and Python demos, and a demo supplied directly by Waveshare support.

Electrical measurement confirms BUSY (GPIO24) is **actively driven low**, not floating.

Hardware tested

Component	Units tried
10.85" (G) panel	2 — original + RMA replacement
10.85" e-Paper HAT+ (G)	2 — original + RMA replacement (shipped together)
Raspberry Pi	3 — Zero 2 W (BCM2710A1), Zero W (BCM2835), and 3 Model B Rev 1.2 (BCM2837)
Power supply	2 — dedicated 5V/2.5A adapter, and powered USB hub
Cabling	Both the factory interposer + ribbon assembly, **and** panel FPC connected directly to the HAT (as suggested by Waveshare support)

Interface Config switch confirmed at position **0 (4-line SPI)** throughout.

Software stacks tried — all fail identically

1. **Waveshare Python `epd10in85g.py`** + `DEV_Config_*.so` (bit-banged SPI via ctypes)
2. **Waveshare C demo** — `E-paper_Separate_Program/10.85inch_e-Paper_G/RaspberryPi/c`, built with WiringPi 3.18
3. **Waveshare support-supplied demo** — `RPI.zip`, `EPD_10in85` mono driver, WiringPi backend
4. **Raw Python** — `spidev` + `RPi.GPIO`, replicating the documented init sequence manually
5. **Third-party pure-Python driver** — hardware SPI on `/dev/spidev0.0` and `/dev/spidev0.1` with kernel chip-select, `gpiozero` for GPIO, no compiled library at all

Stacks 1–4 use manual CS toggling and bit-banged SPI through the precompiled `.so`. Stack 5 uses an entirely different architecture (kernel hardware SPI, hardware CS). All five produce the same result.

Reproduced on a clean install. To rule out accumulated system state, a fresh SD card was imaged with Raspberry Pi OS Lite (32-bit) and provisioned with nothing beyond the minimum: `git`, `build-essential`, WiringPi, and a fresh clone of `github.com/waveshare/e-Paper`. No project code, no Python, no vendored libraries. Waveshare's own `EPD_10in85g_test` demo, built unmodified from that clone, hangs at the identical point:

```
Debug: EPD_10in85g_test Demo
set wiringPi lib success !!!
Debug: e-Paper Init and Clear...
Debug: e-Paper busy
Debug: e-Paper busy release
Debug: e-Paper busy
[hangs indefinitely]
```

Observed behaviour by driver

(G) driver (`epd10in85g.py`) — `ReadBusyH()` waits while BUSY == 0:

```
EPD init...
bcm2835 init success !!!
e-Paper busy H
e-Paper busy release      <- first wait, after Reset(), returns immediately
e-Paper busy H            <- second wait, after command 0x04 (POWER_ON)
[hangs indefinitely]
```

Mono driver (`EPD_10in85`, from support) — `ReadBusy()` waits for BUSY == 1:

```
Debug: EPD_10in85_test Demo
set wiringPi lib success !!!
Debug: e-Paper Init and Clear...
Debug: e-Paper busy
[hangs indefinitely]
```

Both are consistent with BUSY being held in the busy state permanently after power-on is commanded.

Electrical measurements

HAT ID EEPROM reads correctly — the Pi communicates with the board:

```
$ cat /proc/device-tree/hat/product /proc/device-tree/hat/vendor
Waveshare 10.85inch e-Paper HAT+ (G)
WAVESHARE
```

SPI enabled, both devices present:

```
$ ls /dev/spidev*
/dev/spidev0.0  /dev/spidev0.1
```

BUSY (GPIO24) is actively driven low. Read directly from SoC registers with `pinctrl`, no library in between:

```
$ pinctrl get 24
24: ip    -- | lo // GPIO24 = input
```

Commanding an internal pull-up does not change it:

```
$ pinctrl set 24 ip pu; pinctrl get 24
24: ip    -- | lo
```

Control test on an unused pin confirms pull configuration works on this board:

```
$ pinctrl set 22 ip pu; pinctrl get 22
22: ip    -- | hi
$ pinctrl set 22 ip pd; pinctrl get 22
22: ip    -- | lo
```

So GPIO22 follows the pulls; GPIO24 does not. BUSY is being held low by the hardware, not floating.

(Note: the `--` pull field is expected — BCM2835 pull registers are write-only and cannot be read back.)

Raspberry Pi power is clean throughout:

```
$ vcgencmd get_throttled
throttled=0x0
$ vcgencmd measure_volts core
volt=1.3500V     (varies 1.20-1.35V with load - normal DVFS)
```

No undervoltage detected at any point, on either power supply.

One successful partial refresh — not reproducible

On a single occasion, the official C demo (`EPD_10in85g_test`, built with WiringPi) ran on a Raspberry Pi 3 Model B and the panel **visibly refreshed**, flashing through its colour passes as expected.

That refresh did not complete. It left a faint but permanent darker band across roughly the bottom 30% of the panel — e-ink retains an image only where the particles were actually driven, so the refresh began and stopped partway through.

It has never repeated. Since then, in the identical configuration (same Pi 3B, same power supply, same cable, same interposer, same binary, recompiled), the demo has been run more than a dozen times with full power cycles between attempts and hangs at the same point every time: the busy wait following command `0x04` (POWER_ON).

Hardware that works once, produces a partial result, and then fails consistently is the behaviour this report is ultimately asking about.

Instrumented trace of the init sequence

An instrumented version of `EPD_10in85g_Init()` was written, replicating the sequence exactly (CS asserted and released around every byte, 20/10/20 ms reset) while sampling BUSY at every stage. Output:

```
DEV_Module_Init() ok
    [after module init          ] BUSY=0
-- settling 3s after PWR --
    [after 3s settle            ] BUSY=0
-- reset (20ms hi / 10ms lo / 20ms hi) --
    [RST high                   ] BUSY=1
    [RST low                    ] BUSY=0
    [RST high again             ] BUSY=1
  waiting on BUSY (post-reset), starts at 1, timeout 15s
  -> released after 0.00s (0 transitions)
```

```
-- register configuration --
    [after 0x4D                   ] BUSY=1
    [after 0x00 (panel setting)   ] BUSY=1
    [after 0x06 x2 (booster)      ] BUSY=1
    [after 0x50                   ] BUSY=1
    [after 0x61 (resolution)      ] BUSY=1
    [after 0x65                   ] BUSY=1
    [after 0xE0/E3/E5/E9          ] BUSY=1
-- POWER_ON (0x04) --
    [immediately after 0x04       ] BUSY=1
  waiting on BUSY (power-on), starts at 0, timeout 45s
  -> TIMEOUT after 45s, BUSY still 0 (0 transitions)
```

What this establishes:

1. **The panel is alive and responding to the reset line.** BUSY tracks RST exactly — asserted while held in reset, released when reset lifts.
2. **SPI communication reaches the controller.** The full register sequence is accepted with BUSY remaining idle throughout.
3. **The panel accepts POWER_ON and acknowledges it.** BUSY transitions 1 → 0 within milliseconds of command `0x04`, which is the controller asserting busy as expected.
4. **The power-up never completes.** BUSY then remains at 0 for 45 seconds with **zero transitions** — no oscillation, no partial recovery. The controller begins its power-on and the bias generation never reaches its target.

The failure is therefore isolated to the panel's internal power-up following `0x04`, with communication and reset both verified working.

Reset timing (per Waveshare FAQ) — tested, no effect

The wiki FAQ entry *"Why is the BUSY pin always busy?"* suggests shortening the RST low period, noting that the power-off switch in the driver circuit can cause the board to lose power if reset is held low too long.

This was tested across six durations. After `module_init()` (PWR asserted), BUSY reads **ready** in every case:

```
RST low  2000us -> BUSY [1]
RST low  1000us -> BUSY [1]
RST low   500us -> BUSY [1]
RST low   200us -> BUSY [1]
RST low   100us -> BUSY [1]
RST low    50us -> BUSY [1]
```

Reset is therefore working correctly at any timing, and this FAQ item does not apply. The panel powers up, responds to reset, and reports ready. The failure is specifically that it never completes `0x04` (POWER_ON).

What has been ruled out

- Panel defect (2 units)
- HAT defect (2 units)
- Raspberry Pi model and SoC generation (Zero 2 W, Zero W, and 3 Model B — three SoC generations, identical failure)
- OS and architecture (64-bit and 32-bit Raspberry Pi OS)
- Driver software (5 independent implementations, including Waveshare's own)
- SPI configuration (enabled, both spidev nodes present)
- Interface Config switch position (verified at 0 / 4-line SPI)
- Cable routing (factory interposer assembly, and direct panel-to-HAT connection)
- Cable seating and orientation (reseated and inspected multiple times)

- Reset timing (six RST low durations from 2000µs down to 50µs, per the wiki FAQ — BUSY reports ready in all cases)
- Power supply (four sources, including a 5.1V regulated unit verified at `throttled=0x0` — no undervoltage or throttling — which still fails identically)
- GPIO backend and SPI clock rate (the C demo was built against both WiringPi and bcm2835; both bit-bang SPI in software at different clock rates, and both hang at the same point)
- Kernel SPI interface enabled vs disabled (`dtparam=spi=on` and off, both tested)

Questions for Waveshare

1. **What does a permanently-asserted BUSY after command `0x04` (POWER_ON) indicate?** This is the precise failure point. Per the instrumented trace above, the panel acknowledges the command by asserting BUSY within milliseconds, then holds it for 45+ seconds with zero transitions. Reset and SPI communication are both verified working immediately beforehand.
2. **What voltages should be present at the HAT's breakout header (`VCC`, `PWR`) during a successful power-on**, so they can be verified with a multimeter?
3. **What should the panel-side bias rails (VGH, VDH, VDD, VDHR, VCOM — labelled on the panel FPC) read during POWER_ON?**
4. **What supply current does the HAT require?** The failure occurs at the highest-current step of the sequence, so supply capacity was a leading hypothesis. It has since been tested and eliminated on the Pi side: a Raspberry Pi 3 Model B on a 5.1V regulated supply reports `throttled=0x0` — no undervoltage or throttling at any point since boot — and still fails identically at POWER_ON from a cold start.

   Four supplies were tried in total (5V/2.5A adapter, powered USB hub, 5.2V/2.4A Apple 12W, and a 5.1V/2.5A regulated supply). Only the last produces a clean `throttled=0x0`, and the failure is unchanged.
5. **Is there a known erratum** for this panel/HAT combination, or a required init-sequence change not reflected in the published demo code?

Environment

- Raspberry Pi Zero W (BCM2835), Raspberry Pi OS 32-bit (Trixie), kernel 6.18.34+rpt-rpi-v6
- Previously: Raspberry Pi Zero 2 W, Raspberry Pi OS 64-bit
- WiringPi 3.18 (maintained fork) for C builds
- SPI enabled via `dtparam=spi=on`